

PRIMA ESERCITAZIONE:

INDUZIONE STRUTTURALE

Si deve dim. una Prop. su un Insieme definito Induttivamente

① **Definiamo** RBTree come:
$$\begin{cases} \text{foglia} \in \text{BTree} \\ T_1 \in \text{RBTree} \text{ e } T_2 \in \text{Tree} \Rightarrow \text{Branch}(T_1, T_2) \in \text{BTree} \end{cases}$$

Definiamo:

$n: \text{BTree} \rightarrow \mathbb{N}$ che conta #Nodi Nell' Albero

$$n(\text{foglia}) = 1, \quad n(\text{Branch}(T_1, T_2)) = 1 + n(T_1) + n(T_2)$$

Definiamo: $h: \text{BTree} \rightarrow \mathbb{N}$ Calcola altezza (max Cammino)

$$\begin{cases} h(\text{foglia}) = 0 \\ h(\text{Branch}(T_1, T_2)) = 1 + \max(h(T_1), h(T_2)) \end{cases}$$

Messimo

DIMOSTRARE CHE $\forall T \in \text{BTree} : n(t) \leq 2^{h(t)+1} - 1$ (*)

~~///~~ Base, dimostro per le foglie che vale (*)

$$T = \text{foglia} \Rightarrow \begin{aligned} n(t) &= 1 \\ h(t) &= 0 \end{aligned}$$

$$\text{Calcolo (*) per foglia} \Rightarrow 2^{0+1} - 1 = 1$$

ed Infatti è vero visto che $n(t) \leq 1$

Il Passo, Aggiungo il Branch e suppongo I.I. che $n(t) \leq 2^{h(t)+1} - 1$
e $n(t_2) \leq 2^{h(t_2)+1} - 1$

$$T = \text{Branch}(T_1, T_2)$$

Calcolo $n(t) = 1 + n(t_1) + n(t_2)$

$$h(t) = 1 + \max(h(t_1), h(t_2)) \quad \Rightarrow h$$

Applico I.I.

$$n(t) = 1 + n(T_2) + n(T_1) \leq \cancel{1} + 2^{h(T_1)+1} - 1 + 2^{h(T_2)+1} - 1$$

Per DEFINIZIONE so che: $h \geq h(T_1) + 1$ ed Allora Posso Sostituire

$$h \geq h(T_2) + 1$$

$$\leq 2^h + 2^h - 1$$

$$= 2^{h+1} - 1$$

Trucco è stato fare la SOSTITUZIONE h

Sullo Stesso Insieme definiamo:

$L: BTree \rightarrow \mathbb{N}$ che conta le foglie

$$\begin{cases} L(\text{foglia}) = 1 \\ L(\text{Branch}(T_1, T_2)) = L(T_1) + L(T_2) \end{cases}$$

va dimostrato che $L(T) \leq 2^{h(T)} \quad \forall T \in BTree$

BASE $\Rightarrow T = \text{foglia}$ e $L(T) = 1$ e $h(T) = 0$

Trovo che $1 \leq 2^0 = 1$ che è VERO

PASSO $\Rightarrow$ I.I. $L(T_1) \leq 2^{h(T_1)}$ e $L(T_2) \leq 2^{h(T_2)}$

con $T = \text{Branch}(T_1, T_2)$ devo dimostrare $L(T) \leq 2^{h(T)}$

$$L(T) = L(T_1) + L(T_2) \underset{\text{I.I.}}{\leq} 2^{h(T_1)} + 2^{h(T_2)}$$

Prendo $h = \max(h(T_1), h(T_2)) \Rightarrow h(T_1) \leq h$ e $h(T_2) \leq h$

$$\begin{aligned} &| \\ &\leq 2^h + 2^h = 2^{h+1} \end{aligned}$$

è proprio UGUALE ad $2^{h(T)}$

Visto che la def era: $h(t) = 1 + h$

Insieme $EXPR$ Aritmetiche definite Induttivamente

$$\begin{cases} e_1, e_2 \in EXPR & e_1 + e_2 \in EXPR \\ & e_1 - e_2 \in EXPR \\ n \in EXPR \end{cases}$$

Def. $Val: EXPR \rightarrow \mathbb{N}$ Numero di Numeri n presenti in una e

$$\begin{cases} Val(n) = 1 \\ Val(e_1 + e_2) = Val(e_1) + Val(e_2) = Val(e_1 - e_2) \end{cases}$$

Def. $Op: Exp \rightarrow \mathbb{N}$ #operatori

$$\begin{cases} Op(n) = 0 \\ Op(e_1 + e_2) = 1 + Op(e_1) + Op(e_2) = Op(e_1 - e_2) \end{cases}$$

DIMOSTRARE che $Val(e) = Op(e) + 1 \quad \forall e \in EXP$

~~///~~ BASE $\Rightarrow e = n$ e Quindi

$$\begin{cases} Val(e) = Val(n) = 1 \\ Op(e) = Op(n) = 0 \\ 1 = 0 + 1 \quad \text{ed } \bar{e} \text{ verificato} \end{cases}$$

~~///~~ PASSO $\Rightarrow e = e_1 + e_2$

$$\begin{aligned} \text{I.I.} &\Rightarrow Val(e_1) = Op(e_1) + 1 \\ &Val(e_2) = Op(e_2) + 1 \end{aligned}$$

Per def. $Val(e) = Val(e_1) + Val(e_2) \stackrel{J.I.}{=} Op(e_1) + 1 + Op(e_2)$

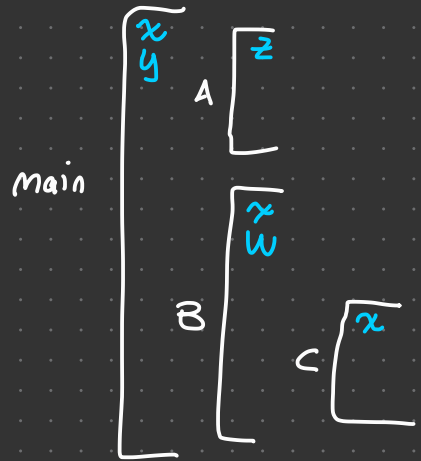
$= Op(e_1) + Op(e_2) + 1 + 1$

$\stackrel{\text{dalla definizione di } Op}{=} Op(e) + 1$

CATENA STATICA:

Viene fornito il Codice, dice Cosa Viene Calcolato. Si Costruisce la Catena Statica (Calcolando K) e Anche la RISOLUZIONE dei RIFERIMENTI non Locali. In caso DINAMICO Mostra l'EVOLUZIONE della CRT.

```
int x=1;
int y=2;
void A() { int z = x+y;
void B() { int x=2;
           int w=4;
           void C() { int x = y+w;
                     A();
                   }
           y=x+y;
           C();
        }
      B(); }
```



①

$$Sd(Main) = 0$$

$$Sd(A) = Sd(B) = -1$$

$$Sd(C) = 2$$

Main $\rightarrow$ B $\rightarrow$ C $\rightarrow$ A

② SCOPING STATICO

$x = 1$
$y = 2$

MAIN Chiama B

link dinamico

$x = 1$
$y = 2$

x	2
w	4

link statico

$$K_B = Sd(Main) - Sd(B) + 1 = 0$$

$$CS(B) = main$$

Eseguendo B ho: $x = w + y$

locale = 2

devo Calcolare N_y (di Quanto
devo Risolvere
Catena Statica)

Più Vicina che definisce y

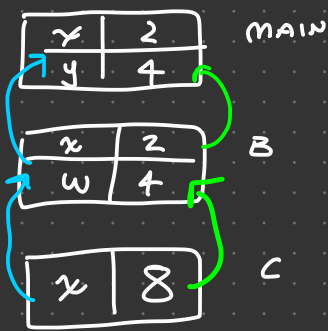
$$N_y = -Sd(Main) + Sd(B)$$

Chi lo usa

$$= 1$$

Risolvere di 1 Per Trovare y
Correttamente [MAIN]

Chiamata di C



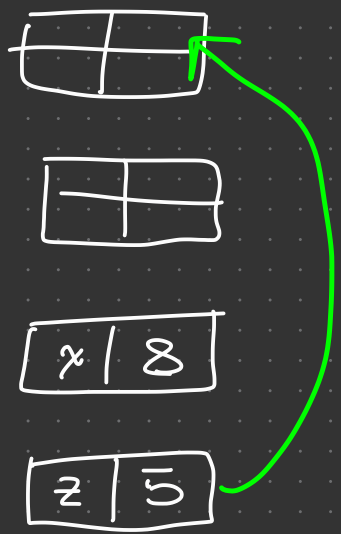
$K_C = Sd(B) - Sd(C) + 1 = 0$
 $CS(C) = B$

$x = y + w = 8$

$N_w = Sd(C) - Sd(B) = 1 \text{ (Blocco Sopra)} \rightarrow 4$

$N_y = Sd(C) - Sd(A) = \text{Main} \rightarrow 4$

Chiamata di A:



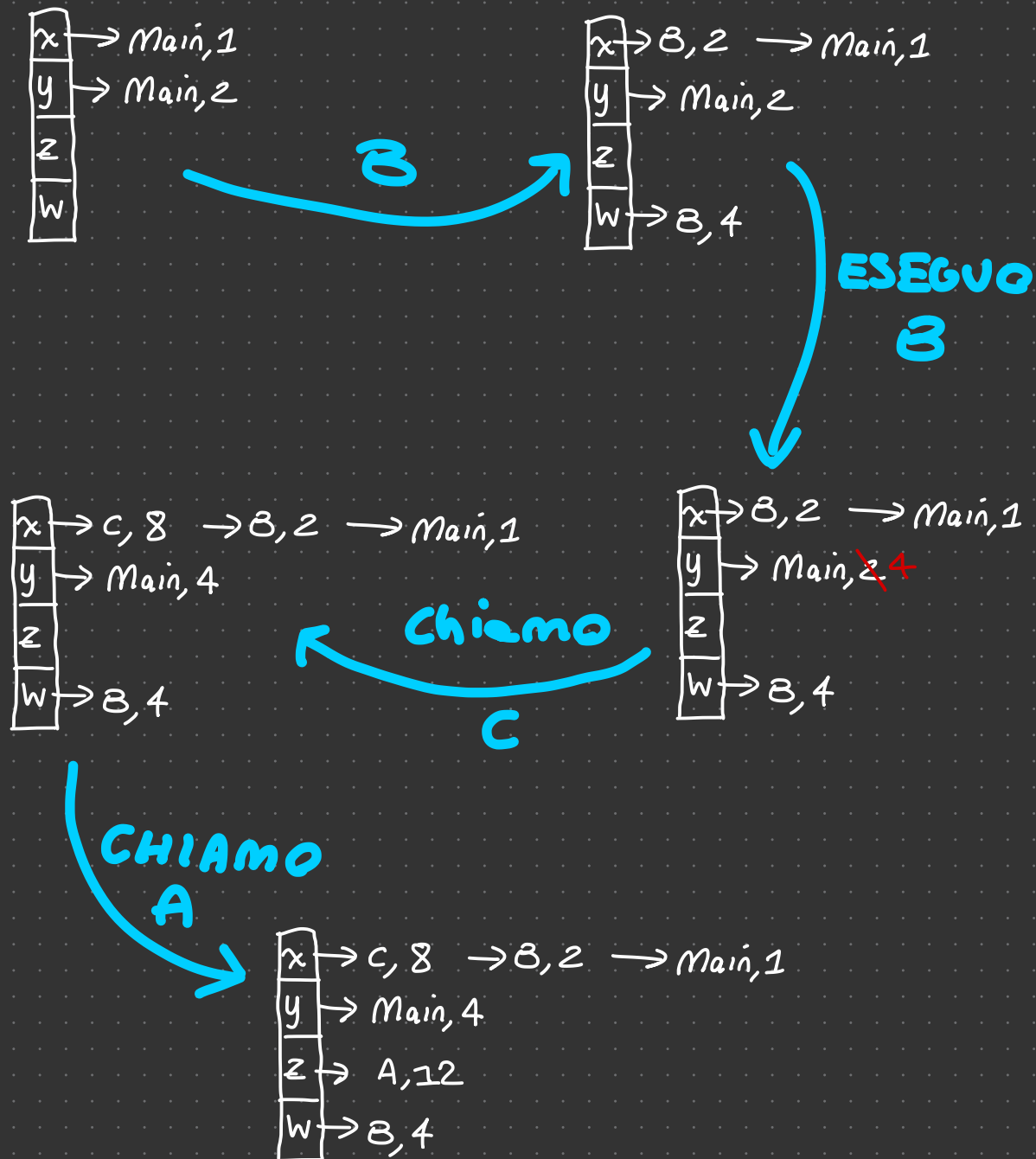
$K_A = Sd(C) - Sd(A) + 1$
 $= 2$

$CS(A) = CS(CS(C)) = CS(B) = \text{Main}$

$z = x + y$
locale (pointing to z) *Sono nel MAIN* (pointing to x+y)
 $= 1 + 4 = 5$
 $N_x = N_y = Sd(A) - Sd(\text{MAIN}) = 1$

SCOPING DINAMICO

Uso CRT Per descrivere



Se fz. Terminasse Prima NON ricopiarle ed Eliminare i Valori che NON Sono più Attivi

RICORSIONE:

Sapere CONCETTI $\Rightarrow$ RICORSIONE e RICORSIONE DI CODA

Fare Simulazione Su Esempio e Trasformarlo in Code

```
lista function (list ~ list) {  
  if (len(list) == 0) Then return list  
  else  
    return concat (function (cdr (list)), car (list))  
}
```

Eseguo su [3,6,9] $\leftarrow (9,6,3)$
 $\hookrightarrow$ concat (function ((6,9), 3) $\leftarrow (9,6)$
 $\hookrightarrow$ concat (function (9, (6)) $\leftarrow (9)$
 $\hookrightarrow$ concat (function (), 9)
 $\downarrow$
()

$\hookrightarrow$ funzione fa il REVERSE della LISTA

Trasforma RICORSIONE IN CODA

```
lista function (lista list) { function RC (list, ()) }
```

```
lista function RC (lista list, lista res) {  
  if (len(list) == 0)  
    return res;  
  else  
    return function RC (cdr(list), concat (car(list), res))  
}
```

function (3, 6, 9)

↳ function RC ((3, 6, 9), ())

↳ function RC ((6, 9), (3))

↳ function RC ((9), (6, 3))

↳ function RC ((), (9, 6, 3))

↓
lista RISULTANTE